PDScope: Breaking the Prefill-Decode Seesaw for Memory-Bound MoE Inference

Enda Yu
National University of Defense Technology
Changsha, China
yuenda@nudt.edu.cn

Dezun Dong
National University of Defense Technology
Changsha, China
dong@nudt.edu.cn

Zhaoning Zhang
National University of Defense Technology
Changsha, China
zhangzhaoning@nudt.edu.cn

Xiaojian Yang
National University of Defense Technology
Beijing, China
yangxj@nudt.edu.cn

Weiling Yang
National University of Defense Technology
Changsha, China
w.yang@nudt.edu.cn

Shaonan Ma
Qiyuan Laboratory
Beijing, China
mashaonan@qiyuanlab.com

Dongsheng Li
National University of Defense Technology
Changsha, China
dsli@nudt.edu.cn

Yongwei Wu
Qiyuan Laboratory
Beijing, China
wuyw@tsinghua.edu.cn

Xiangke Liao
National University of Defense Technology
Changsha, China
xkliao@nudt.edu.cn

Abstract

Local deployment of MoE-based LLMs on CPU-GPU nodes is attractive for cost and privacy reasons, yet limited GPU memory forces most experts into CPU memory. Prefill is then compute-bound and benefits from aggressive GPU loading, while decode is I/O-bound and requires conservative PCIe transfers. These demands cannot be amortized under low batching—typical of local assistants and document tools—forcing phase-agnostic schedulers into a seesaw: favoring prefill's aggressive loading injects unmaskable I/O that stalls decode, while favoring decode's conservative transfers starves prefill's GPU of work. We observe that expert routing is predictable in phase-specific ways, allowing us to decouple the two phases and optimize prediction, scheduling, and execution independently for each phase. PDScope predicts expert activations to guide placement and prefetching, adapts these policies to each phase's bottleneck via cross-layer cost modeling, and compiles decisions into overhead-free pipelines using a static computation graph, cache prefetching, and zero-copy asynchronous I/O. On mainstream MoE models, PDScope achieves 2.5–10.3× higher prefill throughput and 2.1–6.1× higher decode throughput than existing systems, showing that phase-aware co-design unlocks efficient MoE inference under memory constraints and extends naturally to multi-batch settings.

Keywords: Mixture-of-Experts; Offloading; LLM Inference

1 Introduction

The sparse activation of MoE models [5, 24, 34] makes them ideal for local deployment on CPU-GPU servers in cost-sensitive or privacy-prioritizing scenarios. However, limited GPU memory forces most expert parameters to reside in CPU memory, causing the two inference phases to exhibit drastically different bottleneck characteristics.

The prefill phase processes input tokens in parallel, with performance determined by the fraction of tokens processed on the GPU—higher GPU computation translates to greater throughput. The decode phase generates tokens autoregressively, with performance dominated by PCIe transfer latency [20, 25, 27, 30]—lower I/O overhead means reduced latency.

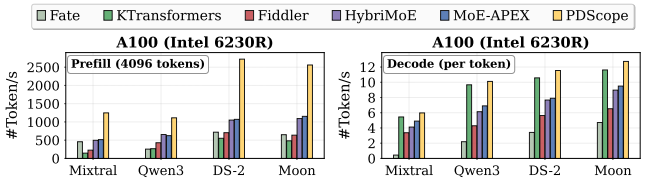


Figure 1. Throughput comparison of prefill (4096 tokens) and decode (per token) on the A100 platform (§5.1).

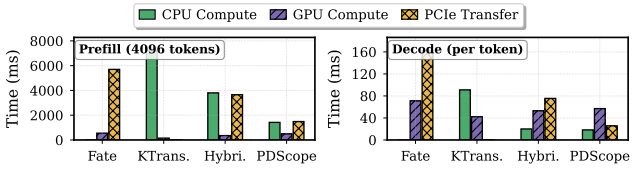


Figure 2. Breakdown of inference overhead for DeepSeek-V2-Lite on the A100 platform. Baselines exhibit a seesaw: Fate-style on-demand loading boosts prefill GPU compute but spikes decode PCIe overhead; kTransformers-style static placement tames decode I/O at the cost of idle prefill GPU. PDScope breaks this trade-off by balancing both metrics.

These two phases place conflicting demands on system resources: prefill benefits from aggressive expert loading to maximize GPU utilization, while decode requires conservative I/O strategies to avoid transfer bottlenecks. This trade-off creates an inherent seesaw effect: optimizing for one phase inevitably degrades the other, as any static resource allocation cannot satisfy both phases simultaneously.

Existing systems explore different trade-off points with unified scheduling logic. Static placement strategies [2, 10, 16, 23] (e.g., kTransformers [2]) keep hot experts (those processing more tokens) resident on the GPU, prioritizing decode stability but leaving GPU compute underutilized during prefill. On-demand loading strategies [8, 11, 13, 44] (e.g., Fate [9]) fetch experts only when needed, improving prefill throughput but introducing I/O bubbles during decode. Dynamic scheduling strategies [33, 45] (e.g., MoE-APEX [33], HybriMoE [45]) monitor runtime load and adapt expert placement at each layer, but they react to the current layer rather than anticipating the next. As Fig. 1 shows, none of these phase-blind approaches escapes the seesaw: on all evaluated models, the on-demand and dynamic schedulers that lead prefill throughput fall behind in decode, while static placement shows the opposite. A common limitation underlies this dilemma: scheduling decisions are made solely from current runtime state, without exploiting the inherent predictability of expert routing. Without lookahead, prefill schedulers cannot distinguish between hot experts that will be needed soon and those that will not, leading to underfetching of critical experts; decode schedulers cannot avoid unnecessary transfers that do not contribute to future tokens, leading to overfetching and I/O bubbles. This myopia prevents existing systems from breaking the seesaw trade-off.

In contrast, we observe that the two phases are predictable in complementary ways. In prefill, the parallel processing of many tokens provides sufficient context to accurately predict upcoming expert activations across layers; in decode, per-token generation follows consistent patterns that allow for stable I/O planning. Phase heterogeneity is therefore not a disturbance to be smoothed over, but a schedulable resource: instead of forcing both phases into a single scheduling framework, each phase can be modeled, scheduled, and optimized independently at the prediction, scheduling, and execution layers, decomposing the conflicting resource requirements into two separate optimization spaces.

We reframe efficient MoE inference as a phase-adaptive scheduling problem: not how to build a fixed execution flow, but how to dynamically configure its optimal operating point for each phase using predicted routing information. This problem is most pronounced in single-batch scenarios on single CPU-GPU nodes, common in interactive applications like local coding assistants or document processing tools. Here, requests arrive in single or few batches, eliminating inter-request parallelism that could mask resource contention between prefill and decode. Without such parallelism, all phase-agnostic systems face the same dilemma—using a unified scheduling strategy to adapt to two diametrically opposed optimization goals, leading to severe resource contention and pipeline stalls. Optimizing for this worst-case scenario unlocks the potential of local MoE deployment; improvements for low-batch phase conflicts naturally extend to multi-batch settings where phase conflicts are less severe (§5.3).

Building on this insight, we present PDScope[1]—an inference system that treats phase-awareness as a first-class scheduling primitive. Its design includes three key elements that work in concert:

Phase-aware prediction, PhasePer, formalizes expert activation prediction as a phase-specific ranking problem. Exploiting the layer-group effect (§3.1), it achieves 94% Top-6 accuracy on DeepSeek-V2-Lite [4] and Qwen3-30B-A3B [35], providing reliable lookahead for phase-appropriate prefetching. **Phase-adaptive scheduling**, AdaptSched, co-optimizes current layer placement and upcoming layer prefetching through cross-layer cost modeling, adopting aggressive policies during prefill and conservative policies during decode. **Overhead-free execution engine** compiles scheduling decisions into efficient sequences through static computation graph, cache prefetching, and zero-copy I/O co-design, preventing runtime overhead from offsetting phase-aware gains.

As shown in Fig. 2, PDScope instead allocates more computation to the GPU during prefill and minimizes PCIe overhead during decode, achieving more balanced resource utilization than phase-blind schedulers. These gains translate into 2.5–10.3× higher prefill throughput and 2.1–6.1× higher decode throughput than existing systems on mainstream MoE models.

2 Background and Related Work

2.1 Inference of MoE Models

A MoE model consists of stacked layers. For each input token x, a layer processes it as follows: the attention module first outputs a hidden state a. The gating module then computes routing weights w_i and selects the top-k activated experts e_i to process a, yielding the layer output x', which serves as input to the next layer. Formally, this can be expressed as:

$$a = \text{Attention}(x) \quad e_i, w_i = \text{Gate}(a) \quad x' = \sum_{i=0}^{k-1} w_i \cdot \text{Expert}_i(a)$$

A key observation is that once a, e_i and w_i are determined, subsequent expert selection becomes deterministic, governed by the model's fixed parameters. This establishes the theoretical foundation for expert predictability.

[1]Code available at https://github.com/nezha-Anonymous/PDScope.

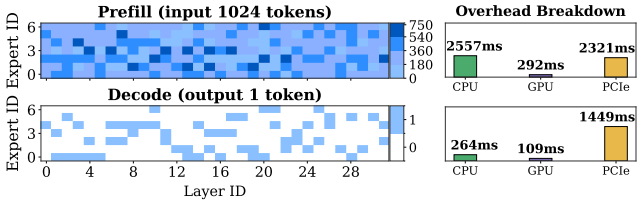


Figure 3. HybriMoE-scheduled Mixtral-8x7B reveals bottlenecks: prefill activates all 8 experts/layer with CPU-dominated overhead, while decode activates 2 experts/layer with PCIe-dominated latency.

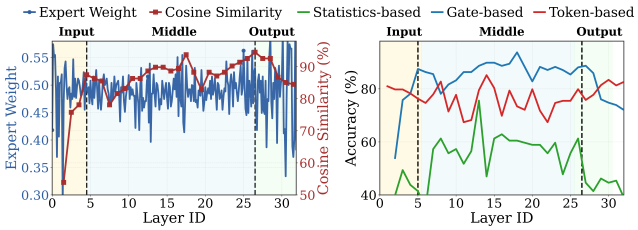


Figure 4. Linking group-wise patterns in routing weight and inter-layer similarity to prediction methods in Mixtral-8x7B.

2.2 Disparity between Prefill and Decode

The resource demand disparity between prefill and decode stems from their fundamentally different token-level parallelism. Fig. 3 quantifies this using Mixtral-8x7B on the A100 platform under a CPU-GPU load-balancing scheduler (HybriMoE [45]). Prefill activates 8 experts per layer, with high token-level parallelism saturating the CPU. Processing more tokens on the GPU alleviates this pressure, yet PCIe transfer latency can stall the GPU. Prefill should therefore prioritize hot experts under PCIe constraints. Decode activates only 2 experts per token; with minimal per-expert computation, loading for load balancing risks making PCIe the bottleneck. Decode should therefore minimize unnecessary I/O. These are two fundamentally different objectives.

2.3 CPU-GPU Inference Related Work

Existing systems improve MoE inference through two complementary techniques: expert prefetching and expert scheduling. **Prefetching** transfers experts in advance to overlap computation with I/O [12, 40, 42]. Its effectiveness hinges on predicting next-layer expert activation. Existing methods fall into three categories: statistics-based methods [3, 8, 20, 43] assume frequently activated experts remain frequent; gate-based methods [9, 18, 29, 45] exploit correlations between adjacent-layer gating inputs; token-based methods [6, 11, 36, 39] reuse historical routing decisions for similar token representations. **Scheduling** decides where to place experts and when to prefetch [28, 31, 32, 37, 38]. Strategies fall into three categories: static placement [2, 10, 16, 23] avoids runtime I/O entirely; on-demand loading [1, 11, 19, 41] keeps experts in CPU memory and loads them only when needed; dynamic scheduling [15, 26, 45] monitors hardware load and dispatches experts to idle devices, though this introduces inspection overhead.

While these methods optimize across the spatial dimension (where to place) and the temporal dimension (when to prefetch), they remain agnostic to the phase dimension (prefill vs. decode). Consequently, their unified logic forces a single operating point that statically compromises between compute-bound and I/O-bound regimes. PDScope distinguishes itself by explicitly exposing the phase axis as a first-class scheduling dimension, enabling two distinct operating points that are independently optimized.

3 Challenges and Motivation

3.1 Expert Prefetching Challenge

Analysis of Mixtral-8x7B [14] reveals a layer-group effect[2] (Fig. 4), where the distributions of routing weights w_i and the similarities of hidden states a across layers cluster MoE layers into input, middle, and output groups with distinct statistical characteristics. As a result, no single heuristic works globally: statistics-based methods suffer from suboptimal overall accuracy; token-based methods falter in the middle groups; gate-based methods, while effective in the middle, fail where inter-layer similarity is low. Exploiting expert activation predictability thus requires layer-group awareness—yet manual or statistical identification of each group's properties is challenging. A learnable predictor, using relevant variables as input features to capture these associations in a data-driven manner, offers a more viable solution.

3.2 Expert Scheduling Challenge

Existing scheduling strategies attempt to handle the prefill/decode disparity with unified logic. As shown in Fig. 1, this seesaw effect is stark: static placement excels in decode but fails in prefill; on-demand loading and dynamic scheduling show the opposite. This section examines dynamic scheduling's failure mechanism: it adapts via real-time load balancing, appearing to be the most promising approach, yet still falls short due to its layer-by-layer greedy nature. Taking HybriMoE [45] as an example, the scheduler monitors per-layer load and greedily assigns hot experts to idle GPUs and cold experts to idle CPUs, pursuing instant balance. This yields suboptimal results in both phases (Fig. 5):

Case 1 (prefill): When only cold experts remain, the scheduler still loads them to the GPU, consuming PCIe bandwidth that should go to prefetching the next layer's hot experts, while the cold experts could execute on CPU without PCIe contention. The current layer achieves local optimum,

[2] *Appendix* §8.7 observes similar patterns in other models.

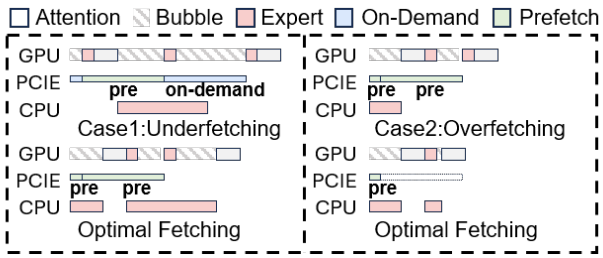


Figure 5. Underfetching in prefill and overfetching in decode caused by layer-by-layer greedy scheduling.

Table 1. PCIe 4.0x16 single-expert transfer latencies (ms).

Model	Benchmark	Prefill	Decode	Theoretical
DeepSeek [4]	1.95	2.78	2.06	0.53
Qwen [35]	1.34	1.88	1.38	0.29
Mixtral [14]	33.5	51.1	33.8	10.67

Benchmark: micro-benchmark without CPU contention; Prefill/Decode: measured during inference with resource competition; Theoretical: minimum latency from PCIe bandwidth limit.

but the next layer stalls-degrading global efficiency. **Case 2 (decode):** When no experts remain in the current layer, the scheduler triggers prefetching for the next layer. If this prefetching does not complete before the next layer's attention and gating finish, it introduces I/O-induced bubbles, exacerbating the decode-phase PCIe bottleneck.

Prefill thus requires aggressive prefetching to maximize tokens processed on the GPU, while decode must avoid unnecessary I/O. Reconciling these conflicting demands requires moving beyond layer-by-layer greediness toward cross-layer coordination that adapts scheduling strategies to each phase.

3.3 Execution Challenge: Overhead Sensitivity

Scheduling gains are bounded by system overhead. Executing a single expert incurs three non-computational costs: CPU cache loading, PCIe transfer, and GPU kernel launch. These overheads differ between prefill and decode, so they cannot be used indiscriminately as scheduling criteria.

In prefill, the CPU is busy, and I/O preparation steps (e.g., pinned memory allocation and DMA setup) compete with computation for CPU resources. Table 1 shows that prefill transfer latency is significantly amplified vs. benchmarks (e.g., DeepSeek-V2-Lite: 1.95ms vs. 2.78ms); the scheduler's real-time advantage is eroded by resource contention.

Decode faces a different predicament: low token-level parallelism prevents overhead amortization. Table 2 reveals a significant gap between computation time and end-to-end runtime for expert computation: overheads from cache loading, kernel launches, and hardware status polling are substantial, particularly on CPU (e.g., Mixtral-8x7B on Intel CPU: 2.88ms compute vs. 4.25ms runtime). Any scheduling query, status polling, or extra transfer directly contributes to end-to-end latency instead of being masked by computation.

Table 2. Single-expert, single-token computation time and end-to-end runtime across hardware platforms (ms).

	Intel 6230R CPU		AMD EPYC CPU		RTX 3090 GPU	
Model	Compute	Runtime	Compute	Runtime	Compute	Runtime
DeepSeek [4]	0.21	0.43	1.5	2.1	0.17	0.29
Qwen [35]	0.16	0.34	1.2	1.64	0.13	0.24
Mixtral [14]	2.88	4.25	9.8	11.2	0.33	0.44

Therefore, scheduling decisions must account for phase-specific overhead characteristics, ensuring that their gains exceed the overhead they introduce. At the same time, the scheduler itself should minimize its own pipeline burden.

4 System Design

4.1 Overview

PDScope is a phase-aware inference system for memory-constrained MoE inference. As shown in Fig. 6, its workflow comprises offline profiling and online execution.

Offline phase. The system profiles the MoE model, collecting per-layer traces (hidden states, routing weights, expert IDs) to train PhasePer. It also builds a cost table mapping token count to runtime cost on CPU and GPU, which underpins AdaptSched's global optimization and adapts to the target platform's bandwidth and compute capability. Hot experts are recorded in a table and preloaded into GPU memory as a static cache to amortize cold-start overhead.

Online phase. PhasePer provides predictions using phase-specific models. AdaptSched consumes these predictions to co-optimize current-layer placement and upcoming-layer prefetching: aggressive prefetching in prefill sustains GPU throughput; conservative I/O in decode minimizes PCIe bubbles. Decisions are compiled into a static computation graph that prescribes all operations and data movements, eliminating per-expert runtime scheduling decisions. Execution follows this graph: a graph-guided cache prefetcher hides memory latency, while a zero-copy asynchronous I/O engine maximizes PCIe bandwidth via pipelined transfers. This runtime stack directly addresses the phase-sensitive overheads analyzed in §3.3, ensuring scheduling decisions translate to performance without adding runtime overhead that would offset gains.

4.2 Learnable Phase-Aware Predictor

4.2.1 Design principle. PhasePer builds on the deterministic mapping from (a, w_i, e_i) to the next layer's expert activation e'_i established in §2.1. It provides separate dedicated predictors for prefill and decode, allowing each to specialize in the statistical patterns of its respective phase.

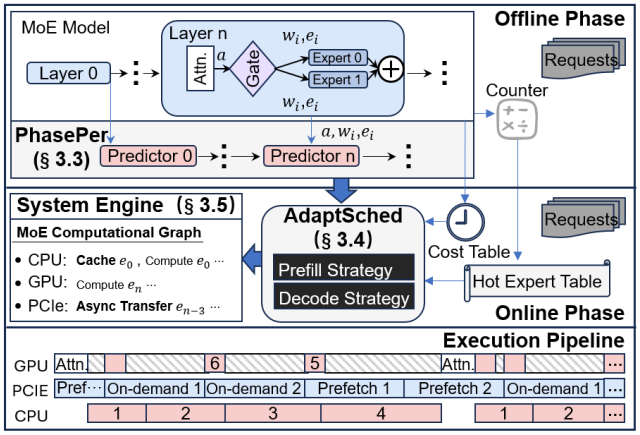


Figure 6. System overview of PDScope.

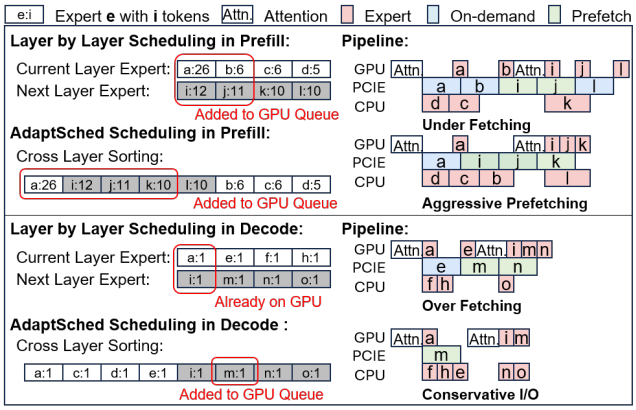


Figure 7. AdaptSched replaces layer-by-layer greediness: prefill eliminates underfetching via aggressive prefetching; decode eliminates overfetching via conservative I/O.

4.2.2 Network architecture.

PhasePer learns the cross-layer function $e'_i = \text{PhasePer}_\theta(a, w_i, e_i)$, mapping the current layer's full state to the next layer's expert activation. This rich input context enables it to capture data-driven selection patterns, yielding more accurate predictions than methods that simply approximate the next layer's gate.

To match the layer-group effect described in §3.1, PhasePer instantiates a dedicated lightweight predictor for each MoE layer. Predictors for layers in the input and output groups, where activation is skewed and features are noisier, use shallow networks to prevent overfitting. For the middle group, where inter-layer features are stable, predictors are deeper and include a residual block modulated by a gating unit that takes a as input to dynamically calibrate the network.

4.2.3 Training strategy.

We collect training data by performing offline inference on 128 ShareGPT [7] batches, each containing a 4096-token prompt followed by 512 generated tokens. Per-layer records (a, w_i, e_i) are organized separately by phase, prefill and decode, to form two dedicated training sets. This phase-isolated data collection ensures that each predictor learns from phase-specific activation patterns. Cross-dataset generalization is evaluated in §5.5.

Loss function. Loss functions are phase-customized. For input/output groups in prefill, a hybrid loss counteracts severe token-level class imbalance and emphasizes hard examples:

$$\begin{aligned} \mathcal{L}_{\text{prefill}} &= \mathcal{L}_{\text{expert}} + \lambda \mathcal{L}_{\text{focal}} \\ \mathcal{L}_{\text{expert}} &= -\frac{1}{N}\sum_{i=1}^{N} \frac{1}{f_i}\,\text{BCE}(y_i, p_i) \\ \mathcal{L}_{\text{focal}} &= -\frac{1}{N}\sum_{i=1}^{N} (1-p_i^{y_i})^{\gamma}\,\text{BCE}(y_i, p_i) \end{aligned} \tag{1}$$

Here, f_i denotes activation frequency of expert i; $y_i \in \{0, 1\}$ indicates ground-truth activation; p_i is the predicted probability; N is number of experts; BCE is the binary cross-entropy loss; and $\lambda, \gamma \geq 0$ are layer-specific hyper-parameters.

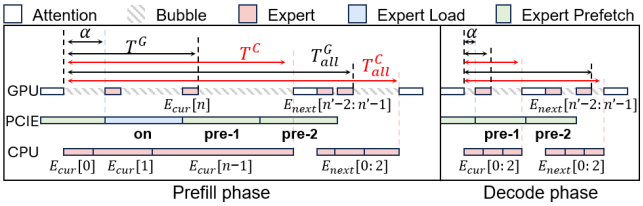


Figure 8. Phase-specific cost model of AdaptSched.

During decode, the expert imbalance is negligible as tokens are processed one by one. Consequently, the loss for the input and output groups reduces to the focal term alone:

$$\mathcal{L}_{\text{decode}} = \mathcal{L}_{\text{focal}} \tag{2}$$

Middle-group predictors in both phases use standard BCE for stable gradients and faster convergence.

Online fine-tuning. PhasePer includes a lightweight online learner. It monitors prediction error on a sliding window of 16 input-group tokens. If the average error exceeds a preset threshold (25% in our implementation), a fine-tuning cycle is triggered using runtime data from 128 prefill and 32 decode tokens, completing within 2.1 ms.

4.3 Phase-Adaptive Cross-layer Scheduler

4.3.1 Design principle.

As shown in Fig. 7, AdaptSched differs from layer-by-layer schedulers by leveraging the predictions of PhasePer for cross-layer visibility[3]. This enables it to co-optimize current-layer placement and next-layer prefetching to maximize PCIe utilization. In prefill, it prioritizes token-heavy experts to maximize tokens processed on the GPU. In decode, it balances on-demand loading, prefetching, and unloading to avoid I/O inefficiencies. Prefetch depth is dynamically calibrated based on predictor confidence to avoid interfering with critical on-demand traffic.

[3]Single-layer lookahead is sufficient; see *Appendix* §??.

4.3.2 Problem modeling.

The scheduler rests on three premises. **(1) Cross-layer sorting:** After excluding experts already on the GPU or being loaded, lists $E_{cur}[0 \dots n]$ and $E_{next}[0 \dots n'-1]$ are formed by sorting experts in ascending order of pending tokens. **(2) On-demand monotonicity:** To maximize I/O benefit, loading decisions are monotonic, if $E_{cur}[i]$ requires loading, so must hotter experts $E_{cur}[i{+}1 : n]$. **(3) Serial I/O:** Transfers are executed sequentially, as single-expert transfers already reach 66–80% PCIe peak (Table 3); adding parallelism risks congestion without faster arrival.

4.3.3 Prefill scheduling.

The scheduler operates in three steps: Step 1 establishes a global resource allocation direction; Step 2 determines the current layer's on-demand loading set; and Step 3 decides the prefetch set.

Step 1: Global queue construction via cross-layer cost modeling. Algorithm 1 merges and sorts experts from the current layer E_{cur} and the predicted next layer E_{next} in ascending order of token count, forming E_{all} in line 9. For the expert at position i in E_{all}, the scheduler evaluates a cross-layer cost model, comparing two completion times depicted in the prefill timeline of Fig. 8:

$$\begin{aligned} T^G_{all} &= \alpha + (n + n' - i + 1) \cdot t_{io} + t_g \\ T^C_{all} &= t_c(E_{all}[0 : i]) + t_{attn} \end{aligned} \tag{3}$$

Here, α is I/O startup latency, t_{io} and t_g are per-expert transfer and GPU computation times, and $t_c(\cdot)$ computes CPU execution time for a set of experts. T^G_{all} estimates the cost if experts $E_{all}[i : n + n']$ are loaded and executed on the GPU, while T^C_{all} estimates the cost of executing experts $E_{all}[0 : i]$ on the CPU (plus attention latency t_{attn}). If $T^G_{all} < T^C_{all}$, experts $E_{all}[i : n + n']$ join the global queue L_{global}. This step, implemented in lines 10-12, selects experts that benefit most from GPU execution from a cross-layer perspective.

Step 2: Layer-local re-evaluation for on-demand loading. Since L_{global} interleaves experts from both layers, directly following it may imbalance the current layer, e.g., all current-layer experts could be assigned to L_{global}, leaving the CPU idle. This can occur due to cross-layer distribution differences or uncertainties in PhasePer's predictions. To correct this, AdaptSched re-evaluates the subsequence of experts belonging to the current layer within L_{global}. For a candidate boundary index i' in E_{cur}, it compares:

$$T_G = \alpha + (n - i' + 1) \cdot t_{io} + t_g \quad T_C = t_c(E_{cur}[0 : i' - 1]) \tag{4}$$

The smallest i' with $T_G < T_C$ finalizes the set L_{on} of experts to be loaded on demand. This step, implemented in lines 14-15, guarantees load balance within the current layer.

Step 3: Confidence-aware prefetching via expected utility. After L_{on} is set, remaining current-layer experts run on CPU. The gap $T_{gap} = T_C - T_G$ represents an I/O bubble, which plus t_{attn} determines maximum hidden prefetches: $f = (T_{gap} + t_{attn})/t_{io}$. Issuing the $|f|$-th prefetch depends on a confidence-aware benefit criterion ξ which computes the expected utility of prefetching by incorporating the predictor's hit rate R_{hit} for high-priority experts:

$$\xi = R_{\text{hit}} \cdot (f - |f| + 1) \cdot t_{io} - (1 - R_{\text{hit}}) \cdot (|f| - f) \cdot t_{io}$$

The first term is the expected gain (weighted by hit rate), and the second term is the expected loss (weighted by miss rate). Prefetching proceeds only if the net expected utility $\xi > 0$. This logic, encapsulated in the PrefetchBenefit function, determines the final prefetch set L_{pre} (lines 19-22).

Algorithm 1: AdaptSched Scheduling for Layer l

Input: Phase $\phi \in \{\text{prefill}, \text{decode}\}$, lists $E_{cur}[0 \dots n]$ and $E_{next}[0 \dots n'-1]$, balance target n_c
Output: Experts to load L_{on}, prefetch L_{pre} and unload L_{un}

1 **if** $\phi = decode$ **then**
2 **if** $n > n_c$ *and* $n' - 1 \le n_c$ **then**
3 **return** $(L_{on}, \emptyset, \emptyset)$; `// Mode a:Load focus`
4 **else if** $n \le n_c$ *and* $n' - 1 > n_c$ **then**
5 **return** $(\emptyset, L_{pre}, L_{un})$; `// Mode b:Prefetch focus`
6 **else if** $n \le n_c$ *and* $n' - 1 \le n_c$ **then**
7 **return** $(\emptyset, \emptyset, L_{un})$; `// Mode c:Unload focus`
  `/* Mode d or prefill: Cross-layer logic */`
8 **Step 1: Build cross-layer global queue:**;
9 $E_{all} \leftarrow \text{MergeAndSortByToken}(E_{cur}, E_{next})$;
10 **for** $i \leftarrow 0$ **to** $|E_{all}| - 1$ **do**
11 **if** $\text{GlobalGpuCost}(i) < \text{GlobalCpuCost}(i)$ **then**
12 $L_{global} \leftarrow E_{all}[i :]$; **Break**;
13 **Step 2: Determine on-demand loading boundary:**;
14 $i' \leftarrow \text{FindSmallestIndex}(L_{global} \cap E_{cur})$ such that $\text{LocalGpuCost}(i') < \text{LocalCpuCost}(i')$;
15 $L_{on} \leftarrow E_{cur}[i' :]$;
16 **Step 3: Decide prefetches:**;
17 $T_{gap} \leftarrow \text{ComputeCpuBubble}(i')$;
18 $f \leftarrow \lfloor (T_{gap} + t_{attn})/t_{io} \rfloor$;
19 **if** $\text{PrefetchBenefit}(f) > 0$ **then**
20 $L_{pre} \leftarrow$ top-f from E_{next};
21 **else**
22 $L_{pre} \leftarrow$ top-$(f-1)$ from E_{next};
23 **return** $(L_{on}, L_{pre}, \emptyset)$

4.3.4 Decode scheduling.

In decode, each token activates top-k experts per layer, making load regular. AdaptSched employs a conservative I/O strategy: it coordinates on-demand loading, prefetching, and unloading based on deviations from the ideal load-balanced state. The optimal CPU-resident expert count n_c minimizes makespan: $n_c = \arg\min \max((k - n_c) \cdot t_g, n_c \cdot t_c)$. This drives four modes (lines 1-7):

Mode a: load focus: If the current layer's CPU-resident expert count is above n_c while the next layer's count is at or below this threshold, the scheduler prioritizes on-demand loading for the current layer, reusing Step 2 logic of Algorithm 1 with a revised cost model:

$$T_G = \alpha + (n - i' + 1) \cdot t_{io} + t_g \quad T_C = t_c(E_{cur}[0 : i']) \tag{5}$$

Unlike the prefill formulation (which keeps boundary experts on CPU to reserve PCIe bandwidth for prefetching), this revised model minimizes current-layer end-to-end latency.

Mode b: prefetch focus. If the current layer's CPU-resident count falls below n_c while next layer's count is above, the scheduler unloads redundant experts and uses PCIe bandwidth for prefetching. Prefetching reuses Step 3 logic of Algorithm 1, subject to an additional constraint:

$$(|f| - f) \cdot t_{io} + t_g < t_c(E_{next}[0 : n' - |f|]) \tag{6}$$

This prevents prefetching from causing next-layer GPU completion time to exceed its CPU-only completion time without boundary prefetching, avoiding decode over-prefetching.

Mode c: unload focus: If both current and next layers' CPU-resident counts below n_c, the scheduler only unloads (backups of all experts are on CPU, so no I/O overhead).

Mode d: cross-layer focus: If both counts are above n_c, the scheduler reuses the full cross-layer logic in Step 1–3, with prefetching subject to the constraint from Mode (b).

4.4 System-Level Optimization Engine

The system engine translates AdaptSched's scheduling decisions into performance by co-optimizing computation orchestration, memory access, and data movement.

4.4.1 Static graph orchestration and runtime optimization. AdaptSched's decisions are compiled into a static computation graph for each layer through a two-phase strategy that decouples scheduling from execution.

Pre-compilation phase. While the runtime executes layer $l-1$, the scheduler uses PhasePer's predictions to pre-compile a draft graph for layer l. This draft prescribes placements and prefetches only for experts with high prediction confidence—top-ranked (hot) and bottom-ranked (cold) experts in E_{next}—decoupling their execution from runtime scheduling. **Recompilation phase.** Once the true gating outputs of layer l become available, PDScope recompiles the complete static graph, resolving all remaining experts using the full AdaptSched logic. The recompiled graph overwrites decisions from the draft, ensuring that prediction errors never propagate to execution.

This pipeline rolls forward: while layer l executes, the scheduler pre-compiles a draft for layer $l+1$. Pre-compilation overlaps scheduling and graph construction overhead with computation, while recompilation corrects for prediction uncertainty using ground-truth information. The runtime then executes each static graph asynchronously. The graph further orchestrates two classes of execution optimizations: CPU-side prefetching of predicted expert weights into the last-level cache before computation begins, and GPU-side kernel launch batching that consolidates all resident and prefetched experts into a single coordinated submission between the attention and MoE phases. This design eliminates per-expert dynamic decision overhead while also reducing the execution latency of expert computation.

Table 3. Expert transfer performance and PCIe bandwidth utilization after AsyncIO optimization in the prefill phase.

	DeepSeek		Qwen		Mixtral	
	Cost (ms)	BW Util.	Cost (ms)	BW Util.	Cost (ms)	BW Util.
PCIe 3.0 ×16	1.39	76%	0.79	72%	26.7	80%
PCIe 4.0 ×16	0.76	70%	0.44	66%	13.98	76%

4.4.2 Zero-copy asynchronous I/O optimizer. The AsyncIO optimizer pre-registers two types of expert weight buffers in GPU memory: a ping-pong buffer for on-demand loading and multiple buffers for prefetching, turning costly dynamic registration into a one-time overhead. This design ensures prefetching for layer $l+1$ never conflicts with on-demand transfers for layer l. To maximize bandwidth, AsyncIO transfers the three matrices of each expert (`gate_proj`, `up_proj`, `down_proj`) concurrently via three independent CUDA streams. All transfers use pinned host memory and asynchronous copies, minimizing CPU intervention. As shown in Table 3, AsyncIO achieves 66–80% PCIe bandwidth utilization, approaching the practical peak.

5 Evaluation

5.1 Experimental Setup

Platform. We evaluate on two hardware platforms: A100 (NVIDIA A100 GPU with dual-socket Intel Xeon Gold 6230R[4] CPUs) and RTX 3090 (NVIDIA RTX 3090 GPU with AMD EPYC 7452 CPU, allocated 16 cores per GPU on a cloud node). Both use PCIe 4.0x16.

Models. The main evaluation uses four BF16 MoE models: Mixtral-8x7B [14], Qwen3-30B-A3B [35], DeepSeek-V2-Lite [4], and Moonlight-16B-A3B [22]. Their key differences are summarized in Table 4. For larger-model scalability, we include DeepSeek-V4-Flash (INT8, 291B) [5] in §5.3.

Baselines. PDScope compares against five state-of-the-art baselines spanning the three paradigms discussed in §2.3: KTransformers[5] [2] pre-allocates experts via static placement with customized AVX-512 kernels; Fate [9] fetches experts on-demand using gate-based prediction; Fiddler [15], HybriMoE [45], and MoE-APEX [33] employ dynamic scheduling with runtime load monitoring, with MoE-APEX using INT4 compression selectively on less important experts. All baselines are evaluated at their individually tuned optimal configuration under the same memory constraints, after three rounds of method-specific warm-up (e.g., kernel warm-up for KTransformers, dynamic hot expert search for HybriMoE).

Memory configuration. We cap GPU memory at 24GB. After allocating non-expert weights and runtime buffers,

[4]Intel Xeon Gold 6230R supports AVX-512 but not AMX.

[5]KTransformers v0.3.2, consistent with their paper.

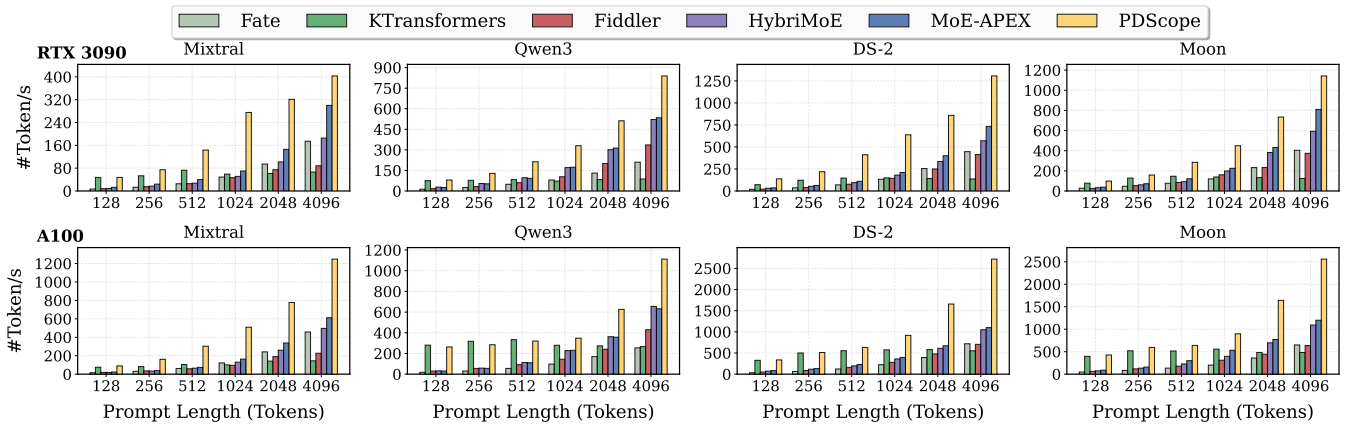


Figure 9. Throughput comparison of PDScope and baseline systems across models and hardware in the prefill phase.

Table 4. Configuration of evaluated MoE models.

	Mixtral	Qwen	DeepSeek	Moonlight
Layers	32	48	26	26
Routed Experts	8	128	64	64
Activated Experts	2	8	6	6
Expert Memory (BF16)	336 MB	9 MB	16.5 MB	16.5 MB

5.8–8.7 GB remains for GPU-resident experts, accommodating each method's allocation strategy (e.g., PDScope's hot expert table, KTransformers' static placement).

Metrics. Throughput is measured separately for the prefill and decode phases[6]. All results are averaged over ten runs.

Datasets. We use ShareGPT [7] with 128–4096 input tokens and 64 output tokens. For out-of-domain evaluation, PhasePer (trained on ShareGPT) is tested on LLaVA [21] (vision-language reasoning) and DPO [17] (preference-tuning).

5.2 End-to-End Performance

5.2.1 Prefill throughput. Fig. 9 presents prefill throughput under varying input lengths across hardware platforms.

Generalization across hardware platforms. Under identical PCIe bandwidth, the systems exhibit distinct hardware dependencies. We compute the geometric mean of A100/RTX 3090 throughput ratios across all (model × length) configurations. The ratios for Fate, Fiddler, HybriMoE, MoE-APEX, and PDScope are approximately 1.67, 1.92, 1.85, 1.72, and 1.88, respectively, indicating similar scaling factors across platforms. In contrast, KTransformers exhibits a ratio of 3.23, highlighting its high sensitivity to CPU performance.

Generalization across model architectures. For Mixtral-8x7B, the relative performance of KTransformers and Fiddler declines when input length exceeds 1024 tokens. KTransformers is limited by its static strategy, while Fiddler struggles to schedule hot experts accurately, highlighting the limitations of static placement and simple dynamic scheduling under high-concurrency, compute-intensive workloads. For models with more but smaller experts, effective CPU-GPU load balancing is more critical, favoring HybriMoE, MoE-APEX, and PDScope. The main difference among these methods lies in I/O transfer efficiency and the hot/cold expert distribution of each transfer, which together determine the total token throughput the GPU can process per unit time.

Scalability across input lengths. KTransformers throughput peaks between 512 and 1024 tokens before declining, indicating inability to adapt to long-prompt compute intensity. Although Fiddler and HybriMoE throughput increases with sequence length, their scaling efficiency lags significantly due to lack of cross-layer coordination: longer inputs activate more experts, increasing I/O overhead, while the widening cost gap between cold and hot experts exacerbates the penalty of suboptimal placement. MoE-APEX suffers from quantization decision overhead under long sequences, resulting in a scaling curve close to that of HybriMoE. In contrast, PDScope exhibits the steepest throughput growth as input length increases, benefiting from cross-layer scheduling and systematic overhead optimization.

In summary, during prefill, PDScope achieves up to 10.3× the throughput of Fate, 8.6× of KTransformers, 5.8× of Fiddler, 4.4× of HybriMoE, and 2.5× of MoE-APEX.

5.2.2 Decode throughput. Fig. 10 presents decode throughput across two hardware platforms. The cross-platform geometric mean ratios are: Fate 1.71×, Fiddler 1.69×, HybriMoE 1.68×, MoE-APEX 1.49×, and PDScope 1.38×. KTransformers exhibits a ratio of 2.86×, due to its CPU kernels optimized for Intel AVX-512, which do not generalize to AMD.

[6]We adopt the same metrics set from KTransformers [2].

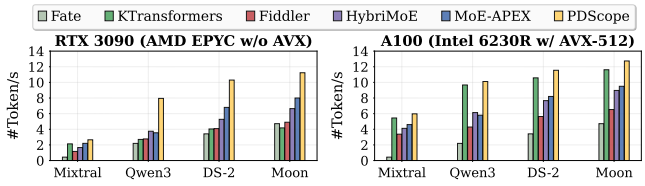


Figure 10. Throughput comparison of PDScope and baseline systems in the decode phase.

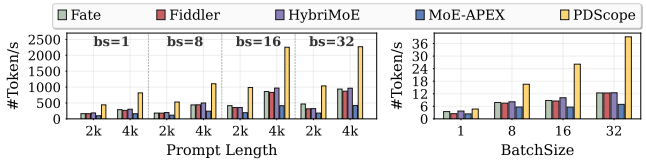


Figure 11. Throughput on DeepSeek-V4-Flash under different batch sizes: (left) prefill; (right) decode.

For Mixtral-8×7B, single-expert I/O cost far exceeds its CPU computation cost; KTransformers has a relative advantage due to its static placement and CPU-optimized kernels, while performance differences among other methods primarily stem from pipeline bubbles from I/O operations. For models with more but smaller experts, dynamic scheduling methods gain some advantage through load balancing, but PDScope further extends its lead via intelligent prefetching and minimal runtime overhead.

In summary, during decode, PDScope achieves up to 6.1× the throughput of Fate, 2.9× of KTransformers, 2.9× of Fiddler, 2.1× of HybriMoE, and 2.1× of MoE-APEX.

5.3 Scalability Analysis

We evaluate PDScope's scalability on the A100 platform using DeepSeek-V4-Flash from two dimensions: model scale and batch scale, as shown in Fig. 11.

Model scalability. KTransformers is excluded as it does not support this model on A100. MoE-APEX exhibits lower performance because it requires storing both full-precision and compressed expert copies on the CPU; enabling pinned memory would exceed the 512GB DRAM capacity, so this I/O optimization is disabled. PDScope achieves 2.82×, 3.06×, 2.69×, and 5.03× prefill throughput improvement over Fate, Fiddler, HybriMoE, and MoE-APEX, respectively, and 1.35×, 1.90×, 1.26×, and 1.97× decode throughput improvement in single-batch settings.

Batch scalability. In multi-batch, all systems enable chunked prefill (chunk size 256) to stay within the 24GB VRAM limit. At batch size 32, PDScope achieves 2.42×, 2.61×, 2.35×, and 5.37× prefill throughput improvement over Fate, Fiddler, HybriMoE, and MoE-APEX, respectively, and 3.16×, 3.17×, 3.14×, and 5.65× decode throughput improvement. Notably, increasing batch size from 16 to 32 yields only modest prefill throughput gains across all systems (e.g., PDScope: 5.2% for 2048-token, 0.7% for 4096-token), indicating that a batch size of 16 already saturates the hardware in this setting.

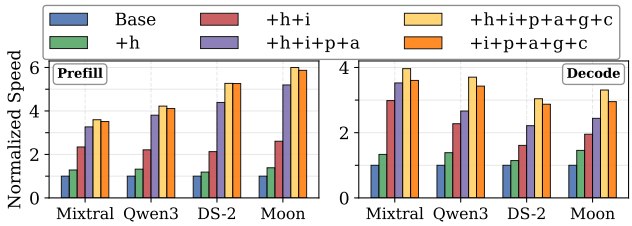


Figure 12. Normalized speed compared to baseline: +h (hot expert table), +i (AsyncIO), +p (PhasePer), +a (AdaptSched), +g (static graph), +c (cache prefetching)

5.4 Ablation Study

To quantify the contribution of each component, we start from Fiddler as the baseline on the A100 platform and incrementally enable PDScope's optimizations. Results are shown in Fig. 12. All reported improvements are incremental gains computed as the ratio of throughput with the component enabled to throughput without it.

Hot expert table (+h). This optimization trades spare GPU memory for reduced on-demand I/O overhead by preloading the most frequently accessed hot experts, delivering 1.16×–1.39× and 1.15×–1.46× throughput improvement in prefill and decode, respectively. **AsyncIO (+i).** This component reduces the effective I/O overhead of expert data transfer, delivering 1.67×–1.88× in prefill and 1.34×–2.24× in decode. **PhasePer & AdaptSched (+p+a).** These core components are evaluated together, as PhasePer's predictions are the direct input to AdaptSched's scheduling decisions; separating them would break this causal chain and underestimate their combined effect. Their synergy delivers 1.39×–2.06× throughput in prefill by exploiting predictable load imbalance. Decode-phase gains range from 1.17× to 1.38× across models. **Static graph & cache prefetching (+g+c).** These system-level optimizations deliver 1.10×–1.20× in prefill and 1.13×–1.39× in decode, with more pronounced gains in decode where fine-grained runtime overheads dominate.

Robustness under memory constraints. Disabling the hot expert table results in only negligible prefill throughput loss (avg. 1.8%) and modest decode loss (avg. 8.2%), reflecting functional overlap with other components. In this configuration, expert weights are streamed directly through the I/O buffers described in §4.4.2, overwriting previous content without explicit eviction.

5.5 Predictor Accuracy

We evaluate PhasePer's generalization on three datasets (ShareGPT, DPO, LLaVA) using 4096-input/512-output samples, after offline training on 16% of ShareGPT. Prediction

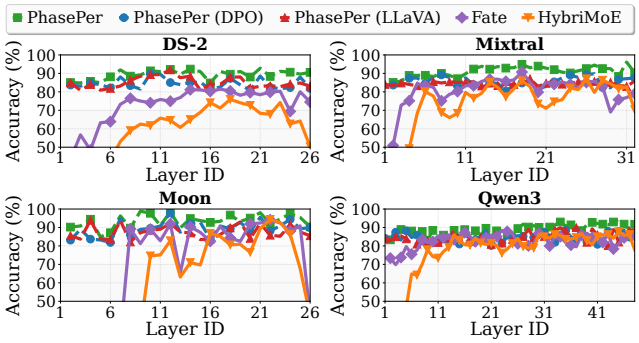


Figure 13. Prefill-phase prediction accuracy of PhasePer across different models and datasets.

Table 5. Performance Evaluation of PhasePer.

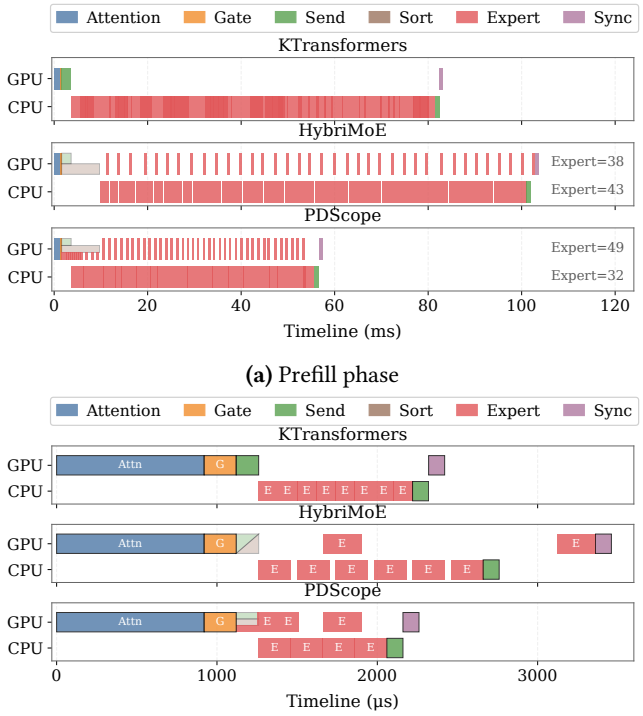
	DeepSeek	Mixtral	Qwen	Moonlight
ShareGPT Top-6	96%	100%	97%	99%
DPO Top-6	92%	99%	94%	95%
LLaVA Top-6	90%	99%	92%	92%
Memory	1.6-4.0 MB	1.2-3.8 MB	1.7-4.2 MB	1.6-4.0 MB
Time	0.48 ms	0.15 ms	0.12 ms	0.48 ms

accuracy is measured as the hit rate against the experts selected by the gating network.

Fig. 13 shows the results. PhasePer's accuracy on out-of-domain datasets (DPO, LLaVA) drops only modestly (avg. 4.8–5.3%) compared to ShareGPT, and never falls below 80% even in low-similarity input/output groups—demonstrating strong generalization to domain shifts. Compared to Fate, PhasePer improves accuracy by 15% to 68.4%. Decode-phase results follow the same trend, with average accuracy 2.3% higher than in prefill due to stronger inter-layer similarity; full decode data are in the *Appendix* §8.2.

We further examine PhasePer's impact on end-to-end performance. Table 5 reports the Top-6 prefetch accuracy for experts ranked by token count, where a prediction is correct if the predicted expert falls within a ±2 position range of the true ranking (e.g., the predicted 6th expert is within the true 4th to 8th). This tolerance-based metric is chosen because AdaptSched's scheduling decisions rely on the relative ordering of expert "hotness" rather than exact rankings; small positional deviations still guarantee the expected prefetching utility. PhasePer's Top-6 hit rate remains above 90%, fully meeting AdaptSched's precision requirement.

In terms of resource usage, PhasePer deploys one micro-predictor per layer, with parameter counts as low as 1.2 MB; middle-layer predictors use slightly deeper architectures, up to 4.2 MB. The forward-pass latency ranges from 0.12 to 0.48 ms; this latency is hidden within GPU idle time during expert transfers, introducing no additional pipeline bubbles.



(a) Prefill phase

(b) Decode phase

Figure 14. Runtime breakdown on Qwen3-30B-A3B (A100).

5.6 Runtime Trace Analysis

To gain a microscopic understanding of the performance differences, we capture runtime execution traces of a single layer of Qwen3-30B-A3B on A100 with 1024 input tokens and 64 output tokens (Fig. 14). Hot expert preloading is disabled to avoid confounding with runtime prefetching.

In prefill, KTransformers executes all expert computation on the CPU, with performance limited by CPU compute capability. HybriMoE, despite its dynamic load-balancing capability to reduce GPU idleness, does not achieve superior performance. In contrast, PDScope's trace shows higher GPU utilization. Computation gaps are smaller because the static graph eliminates the dynamic scheduler's polling overhead and AsyncIO improves I/O transfer performance. A higher proportion of experts are also prefetched per layer, reflecting PhasePer's accurate predictions and AdaptSched's cross-layer coordination. The "Sort"—which encompasses both scheduling decisions and graph construction—takes 8.1 ms and overlaps with 12 GPU expert computations and 4 CPU expert computations, confirming that these operations can be hidden within the pipeline without adding extra delay.

In decode, KTransformers runs efficiently on CPU with optimized kernels and no scheduling overhead. HybriMoE's second expert transfer does not arrive at the GPU at the expected time, failing to achieve ideal load balancing. PDScope improves system performance through three mechanisms:

(1) PhasePer ensures prefetched experts are used, reducing bandwidth waste; (2) AdaptSched's decode-phase scheduling is more precise, avoiding performance loss from excessive I/O; and (3) system optimizations such as kernel launch batching and cache prefetching ensure data readiness before computation begins, reducing computation gaps.

These traces validate PDScope's core claims: phase-adaptive scheduling eliminates greedy underfetching in prefill and aggressive I/O in decode; system-level optimizations translate decisions into performance without runtime overhead.

6 Discussion

This paper targets low-batch inference as the canonical model for studying phase heterogeneity. PDScope does not support the prefill/decode overlap used in distributed inference: on memory-constrained CPU-GPU nodes, prefill bottlenecks shift to CPU and I/O (§2.2)—overlapping decode would only intensify contention. PDScope supports multi-batch phase-consolidated batching (§5.3): AdaptSched reuses the same prefill logic, while decode operates in Mode d, where operations are planned without resource contention.

For platform diversity, the offline cost table calibrates operation latencies, enabling AdaptSched to adapt to different hardware. The full offline phase takes under 30 minutes per model on A100. AdaptSched is resilient to prediction errors: even when PhasePer accuracy drops to 45–70%, prefill throughput decreases by only 6.4–11.3%. Step 2's layer-local re-evaluation prevents cascading errors by falling back to on-demand loading for mispredicted experts. Detailed analysis is in the *Appendix* §**??**.

The hot expert table is derived from the offline phase, a static cache with limited capacity under tight memory. Disabling it in §5.4 has limited impact on performance.

7 Conclusion

This paper identifies phase heterogeneity—the fundamentally different compute and I/O demands of prefill and decode—as the root cause of inefficiency in MoE inference on memory-constrained CPU-GPU nodes. Existing phase-agnostic systems, regardless of their scheduling strategy, inevitably compromise one phase for the other. PDScope demonstrates that explicitly separating the two phases across prediction, scheduling, and execution layers turns this heterogeneity from a liability into a schedulable resource. Our evaluation shows that PDScope achieves up to 10.3× prefill and 6.1× decode throughput over state-of-the-art baselines.

Acknowledgments

This work was supported in part by the New Generation Artificial Intelligence-National Science and Technology Major Project (Grant No. 2025ZD0122202), the National Natural Science Foundation of China (Grant Nos. 62421002 and U24B20151), and the Science and Technology Innovation Program of Hunan Province (Grant No. 2025RC3118). Dezun Dong, Shaonan Ma, Yongwei Wu, and Xiangke Liao hold joint appointments at Qiyuan Lab. Dezun Dong and Zhaoning Zhang are the corresponding authors.

References

[1] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E Gonzalez, Matei Zaharia, and Ion Stoica. 2025. Moe-lightning: High-throughput moe inference on memory-constrained gpus. In *Proc. ACM ASPLOS*. 715–730. https://doi.org/10.1145/3669940.3707267

[2] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. 2025. KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models. In *Proc. ACM SOSP*. 1014–1029. https://doi.org/10.1145/3731569.3764843

[3] Peizhuang Cong, Aomufei Yuan, Shimao Chen, Yuxuan Tian, Bowen Ye, and Tong Yang. 2024. Prediction is all moe needs: Expert load distribution goes from fluctuating to stabilizing. arXiv:2404.16914 https://doi.org/10.48550/arXiv.2404.16914

[4] DeepSeek-AI. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv:2405.04434 https://doi.org/10.48550/arXiv.2405.04434

[5] DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv:2606.19348 https://arxiv.org/abs/2606.19348

[6] Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai Li, and Yiran Chen. 2024. Sida: Sparsity-inspired data-aware serving for efficient and scalable large mixture-of-experts models. *MLSys* 6 (2024), 224–238. https://proceedings.mlsys.org/paper_files/paper/2024/hash/698cfaf72a208aef2e78bcac55b74328-Abstract-Conference.html

[7] Hugging Face. 2024. ShareGPT-V3-unfiltered-cleaned-split. https://huggingface.co/datasets/learnanything/sharegpt_v3_unfiltered_cleaned_split.

[8] Zhiyuan Fang, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. 2025. Klotski: Efficient Mixture-of-Expert Inference via Expert-Aware Multi-Batch Pipeline. In *Proc. ACM ASPLOS*. 574–588. https://doi.org/10.1145/3676641.3716261

[9] Zhiyuan Fang, Xingfan Yu, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, and Fan Yu. 2026. Fate: Fast Edge Inference of Mixture-of-Experts Models via Cross-Layer Gate. In *Proc. ACM WWW*. 5537–5546. https://doi.org/10.1145/3774904.3792527

[10] Georgi Gerganov. 2023. *llama.cpp*. https://github.com/ggerganov/llama.cpp

[11] Xin He, Shunkang Zhang, Yuxin Wang, Haiyan Yin, Zihao Zeng, Shaohuai Shi, Zhenheng Tang, Xiaowen Chu, Ivor Tsang, and Ong Yew Soon. 2026. ExpertFlow: Efficient Mixture-of-Experts Inference via Predictive Expert Caching and Token Scheduling. arXiv:2410.17954 https://doi.org/10.48550/arXiv.2410.17954

[12] Haiyang Huang, Newsha Ardalani, Anna Sun, Liu Ke, Shruti Bhosale, Hsien-Hsin Lee, Carole-Jean Wu, and Benjamin Lee. 2024. Toward efficient inference for mixture of experts. *NIPS* 37 (2024), 84033–84059. https://doi.org/10.52202/079017-2670

[13] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proc. IEEE ISCA*. 1018–1031. https://doi.org/10.1109/ISCA59077.2024.00078

[14] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier,

Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of experts. arXiv:2401.04088 https://doi.org/10.48550/arXiv.2401.04088

[15] Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasikci. 2025. Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models. In *Proc. ICLR*. 56099–56115. https://openreview.net/forum?id=N5fVv6PZGz

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proc. ACM SOSP*. 611–626. https://doi.org/10.1145/3600006.3613165

[17] Xinlu Lai. 2024. The DPO Dataset for Chinese and English with emoji. https://huggingface.co/datasets/shareAI/DPO-zh-en-emoji.

[18] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *Proc. USENIX OSDI*. 155–172. https://www.usenix.org/conference/osdi24/presentation/lee

[19] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating distributed MoE training and inference with lina. In *Proc. USENIX ATC*. 945–959. https://www.usenix.org/conference/atc23/presentation/li-jiamin

[20] Kexin Li, Wenkan Huang, Qinggang Wang, Long Zheng, Xiaofei Liao, Hai Jin, and Jingling Xue. 2025. Diff-MoE: Efficient Batched MoE Inference with Priority-Driven Differential Expert Caching. In *Proc. ACM SC*. 1951–1965. https://doi.org/10.1145/3712285.3759903

[21] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. *NIPS* 36 (2023), 34892–34916. http://papers.nips.cc/paper_files/paper/2023/hash/6dcf277ea32ce3288914faf369fe6de0-Abstract-Conference.html

[22] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. 2025. Muon is Scalable for LLM Training. arXiv:2502.16982 https://doi.org/10.48550/arXiv.2502.16982

[23] LMSYS. 2025. *SGLang*. https://github.com/sgl-project/sglang

[24] Meta. 2025. *Llama 4*. https://www.llama.com/models/llama-4/

[25] Xiurui Pan, Endian Li, Qiao Li, Shengwen Liang, Yizhou Shan, Ke Zhou, Yingwei Luo, Xiaolin Wang, and Jie Zhang. 2025. InstAttention: In-Storage Attention Offloading for Cost-Effective Long-Context LLM Inference. In *Proc. IEEE HPCA*. 1510–1525. https://doi.org/10.1109/HPCA61900.2025.00113

[26] Yudong Pan, Yintao He, Tianhua Han, Lian Liu, Shixin Zhao, Zhirong Chen, Mengdi Wang, Cangyuan Li, Yinhe Han, and Ying Wang. 2026. TriMoE: Augmenting GPU with AMX-Enabled CPU and DIMM-NDP for High-Throughput MoE Inference via Offloading. arXiv:2603.01058 https://doi.org/10.48550/arXiv.2603.01058

[27] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning. In *Proc. ACM SC*. 1–14. https://doi.org/10.1145/3458817.3476205

[28] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *Proc. ICML*. 31094–31116. https://proceedings.mlr.press/v202/sheng23a.html

[29] Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. 2024. Promoe: Fast moe-based llm serving using proactive caching. arXiv:2410.22134 https://doi.org/10.48550/arXiv.2410.22134

[30] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. In *Proc. ACM SOSP*. 590–606. https://doi.org/10.1145/3694715.3695964

[31] Ruslan Svirschevski, Avner May, Zhuoming Chen, Beidi Chen, Zhihao Jia, and Max Ryabinin. 2024. Specexec: Massively parallel speculative decoding for interactive llm inference on consumer devices. *NIPS* 37 (2024), 16342–16368. http://papers.nips.cc/paper_files/paper/2024/hash/1d91d5689e251d27993a3c2182dddcf7-Abstract-Conference.html

[32] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. Hobbit: A mixed precision expert offloading system for fast moe inference. arXiv:2411.01433 https://doi.org/10.48550/arXiv.2411.01433

[33] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2026. MoE-APEX: An Efficient MoE Inference System with Adaptive Precision Expert Offloading. In *Proc. ACM ASPLOS*. 1185–1200. https://doi.org/10.1145/3779212.3790187

[34] Kimi Team. 2025. Kimi Linear: An Expressive, Efficient Attention Architecture. arXiv:2510.26692 https://arxiv.org/abs/2510.26692

[35] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 https://doi.org/10.48550/arXiv.2505.09388

[36] Chengcheng Wang, Haowen He, Liang Zhao, Xiaoheng Deng, Lixin Duan, and Shaohua Wan. 2026. CasMoE: A Cascaded Framework for Efficient MoE Inference on Resource-constrained Devices. In *AAAI*. 26133–26141. https://doi.org/10.1609/aaai.v40i31.39816

[37] Daliang Xu, Wangsong Yin, Hao Zhang, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. 2025. EdgeLLM: Fast On-Device LLM Inference With Speculative Decoding. *IEEE TMC* 24 (2025), 3256–3273. https://doi.org/10.1109/TMC.2024.3513457

[38] Tairan Xu, Leyang Xue, Zhan Lu, Adrian Jackson, and Luo Mai. 2025. MoE-Gen: High-Throughput MoE Inference on a Single GPU with Module-Based Batching. arXiv:2503.09716 https://doi.org/10.48550/arXiv.2503.09716

[39] Fuzhao Xue, Zian Zheng, Yao Fu, Jinjie Ni, Zangwei Zheng, Wangchunshu Zhou, and Yang You. 2024. OpenMoE: an early effort on open mixture-of-experts language models. In *Proc. ICML*. 55625–55655. https://proceedings.mlr.press/v235/xue24c.html

[40] Jinghan Yao, Quentin Anthony, Aamir Shafi, Hari Subramoni, and Dhabaleswar K DK Panda. 2024. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference. In *Proc. IEEE IPDPS*. 915–925. https://doi.org/10.1109/IPDPS57955.2024.00086

[41] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2025. EdgeMoE: Empowering Sparse Large Language Models on Mobile Devices. *IEEE TMC* 24 (2025), 7059–7073. https://doi.org/10.1109/TMC.2025.3546466

[42] Hanfei Yu, Xingqi Cui, Hong Zhang, and Hao Wang. 2025. fMoE: Fine-Grained Expert Offloading for Large Mixture-of-Experts Serving. arXiv:2502.05370 https://doi.org/10.48550/arXiv.2502.05370

[43] Junyi Zhang, Chuanhu Ma, Xiong Wang, Yuntao Nie, Yuqing Li, Yuedong Xu, Xiaofei Liao, Bo Li, and Hai Jin. 2025. PopFetcher: Towards Accelerated Mixture-of-Experts Training Via Popularity Based Expert-Wise Prefetch. In *Proc. USENIX ATC*. 1053–1069. https://www.usenix.org/conference/atc25/presentation/zhang-junyi

[44] Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. 2024. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient moe inference. In *Proc. IEEE ICCAD*. 1–9. https://doi.org/10.1145/3676536.3676741

[45] Shuzhang Zhong, Yanfan Sun, Ling Liang, Runsheng Wang, Ru Huang, and Meng Li. 2025. HybriMoE: Hybrid CPU-GPU Scheduling and Cache Management for Efficient MoE Inference. In *Proc. DAC*. 1–7. https://doi.org/10.1109/DAC63849.2025.11133274

8 appendix

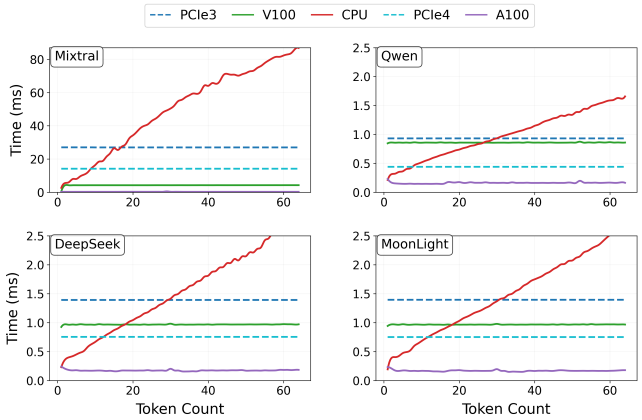
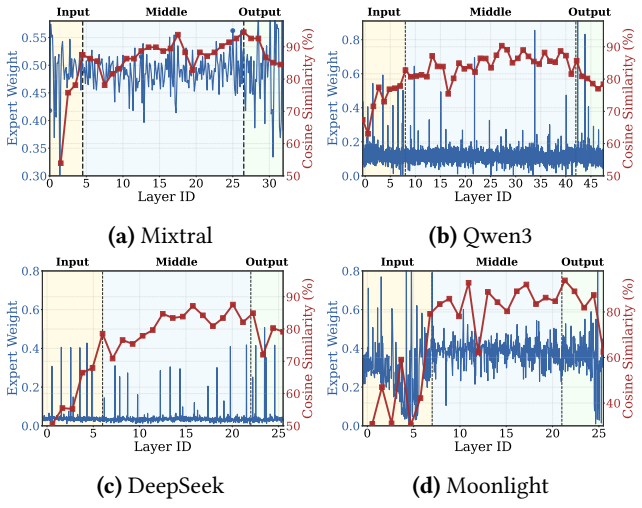


Figure 15. Characterization of expert computation and transfer costs across models and heterogeneous devices.

8.1 Overhead Characterization

Fig. 15 reveals three key observations. First, for both V100 and A100 GPUs, single-expert GPU execution time is significantly lower than PCIe transfer time, indicating that GPU-only solutions are I/O-bound rather than compute-bound. Second, for small-expert architectures (e.g., Qwen), CPU computation delay is lower than GPU startup delay when tokens per expert is below 16; for large-expert models like Mixtral, CPU delay consistently exceeds GPU, determining the break-even point for CPU offloading. Third, as token count increases, GPU's parallel advantage becomes prominent while CPU overhead grows linearly, suggesting GPU should dominate large-volume computation.



(a) Mixtral **(b)** Qwen3

(c) DeepSeek **(d)** Moonlight

Figure 16. Group-wise patterns in routing weights and inter-layer similarity in MoE models.

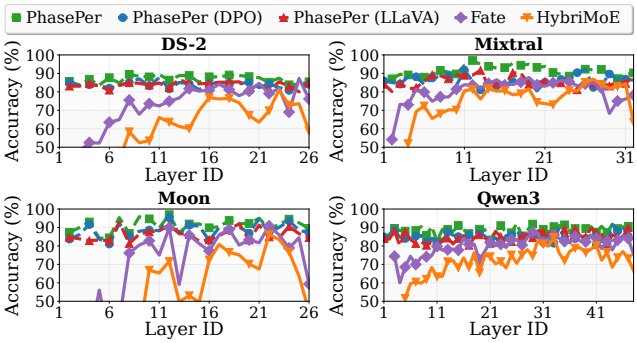


Figure 17. Accuracy of PhasePer in the decode phase.

8.2 Layer-group Effects of Different Models

Fig. 16 shows that all models exhibit significant fluctuation in routing weights within input/output layer groups, indicating strong inter-layer expert correlations in these regions. Within middle layer groups, inter-layer cosine similarity remains consistently high, enabling stable prediction accuracy based on attention-output hidden states. The boundaries between layer groups can be automatically identified by inflection points in cosine similarity: a monotonic increase from low to high similarity during input-to-middle transition, and a corresponding decrease during middle-to-output transition. This observation provides the foundation for PhasePer's adaptive layer-grouping. Decode-phase prediction accuracy for PhasePer is provided in Fig. 17.

8.3 Robustness to Prediction Errors

Although PhasePer demonstrates strong generalization capability in cross-dataset tests (§5.5), we further evaluate the potential impact of predictor accuracy degradation on AdaptSched's performance under extreme conditions. For this purpose, we replaced PhasePer with the prediction strategy used by Fate, which directly predicts the next layer's expert activation based on the previous layer's hidden state passed through the gating module.

Tests on an A100 GPU with an input length of 4096 and output length of 64 showed that prefill throughput for the input/output layer groups decreased by an average of 6.4%–11.3%, while the middle layer group decreased by only 0.9%–2.4%. Combined with the ablation study results (§5.4), these findings demonstrate that even with lower prediction accuracy, AdaptSched's scheduling performance remains significantly superior to layer-by-layer on-demand scheduling strategies without prediction. This robustness under prediction inaccuracy stems from the interaction between AdaptSched's design and the statistical properties of expert activation in sparse MoE models.

Statistical basis of robustness. The fundamental logic can be understood through the relationship between the

random baseline hit rate and the ranking advantage conferred by prediction. We use Mixtral-8×7B as an example. Under the (unrealistic) assumption of random expert selection—ignoring all routing preferences—the probability of any given expert being selected for a token is $2/8 = 25\%$. This is the random baseline hit rate.

Why ranking matters more than perfect accuracy. For a predictor to deliver systematic prefetching benefits, it does not need perfect accuracy. Instead, it only needs to achieve a hit rate for the true "hot experts" (those that ultimately process the most tokens) that is consistently higher than the 25% random baseline. Within a large batch, expert activation can be viewed as multiple independent Bernoulli trials. The value of the predictor lies in generating a ranking of expert "hotness" that is more accurate than random. Even with moderate absolute accuracy, if the predictor's hit rate for the true hot experts exceeds the baseline, those experts are more likely to appear higher in the predicted ranking. Consequently, prefetching I/O operations are statistically more likely to perform "useful work," improving overall throughput.

System-level safeguard. If a hot expert is not prefetched, AdaptSched's layer-local re-evaluation mechanism defaults to on-demand loading for that expert, limiting the latency penalty of any single prediction error. This explains AdaptSched's robustness even with relatively simple predictors.

Implications for different models. For models with small expert counts and low I/O costs, the direct overhead of prediction errors is low, and this mechanism effectively absorbs inaccuracies. For models like Mixtral-8×7B, where each layer has 8 experts under Top-2 activation, most experts have a considerable probability of being used during long-prompt processing. Thus, even prefetching a non-"hottest" expert can often yield partial performance benefits.

8.4 Fine-Tuning Dynamics and Error Propagation

We further analyze two practical aspects that complement the above robustness discussion.

Error propagation prevention. Even in the rare case of consecutive mispredictions, AdaptSched's layer-local re-evaluation (Step 2 in Algorithm 1) acts as a safety net. When PhasePer incorrectly predicts the top-k experts for layer $l+1$, the scheduler may prefetch experts that are never used. However, Step 2 re-evaluates the on-demand loading boundary for the current layer l based on actual token counts, ignoring prefetched but unused experts. For the affected layer $l+1$, the missing experts are fetched on-demand, incurring at most one extra I/O round trip per mispredicted layer. This prevents cascading errors across multiple layers.

Online fine-tuning trigger frequency and latency hiding. The error threshold over a 16-token sliding window is conservatively low. On all evaluated datasets (ShareGPT, DPO, LLaVA), the average interval between fine-tuning triggers exceeds 500 inference steps for prefill and over 1000 steps for decode, because prediction errors are rarely consecutive. The fine-tuning overhead (2.1 ms per trigger) is entirely overlapped with GPU idle periods that naturally occur during expert I/O transfers. In the decode phase, fine-tuning is scheduled during the attention computation of the next token, again achieving full overlap. Consequently, fine-tuning adds zero observable delay to the critical path.

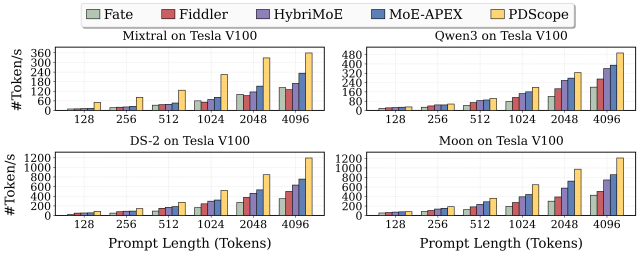


Figure 18. Throughput comparison of PDScope and baseline systems in the prefill phase.

8.5 Throughput on V100

8.5.1 Prefill Throughput. We evaluate on an NVIDIA V100 GPU (PCIe 3.0 x16) with an Intel Xeon Gold 6230R CPU. KTransformers is excluded as it does not support pre-Ampere GPUs. Results in Fig. 18 and Fig. 19 show that relative performance ranking shifts only modestly. All systems show significant performance drop on Qwen3 due to: (1) higher PCIe 3.0 transfer overhead, and (2) higher kernel launch overhead on V100 vs. A100. Consequently, more computation shifts to CPU, limiting overall efficiency.

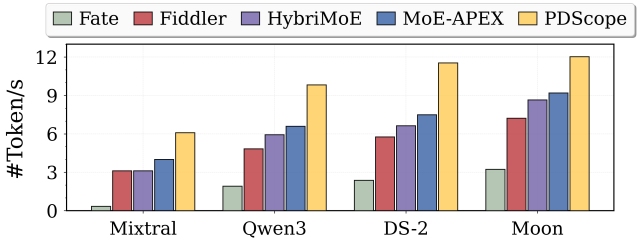


Figure 19. Throughput comparison of PDScope and baseline systems in the decode phase.

8.5.2 Decode Throughput. PDScope maintains its performance advantage on V100, consistent with the analysis in §2.2: decode-phase compute intensity is inherently low, and the core scheduling requirements are avoiding I/O overhead and scheduling-induced latency. Since PDScope's key optimizations—PhasePer, AdaptSched, and system-level optimizations—are not tied to specific hardware capabilities, reduced GPU and PCIe performance has limited impact on its relative gains.

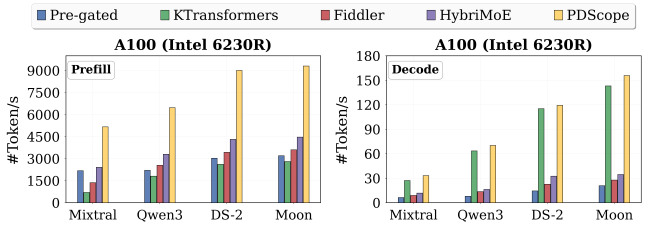


Figure 20. Throughput comparison of PDScope and baseline systems with a batch size of 32.

8.6 Multi-Batch Inference Performance

We complement the scalability analysis in §5.3 with additional observations on A100 at batch size 32 (1024 input, 128 output tokens), using the full set of evaluated models. Results in Fig. 20 confirm that the seesaw effect persists under concurrency: KTransformers excels in decode while Fate excels in prefill, with reversed trends—consistent with the phase-agnostic limitation discussed in §1. PDScope's decode-phase scheduler directly employs Mode d (cross-layer focus) from Algorithm 1, while prefill-phase gains align with the long-context single-batch results in Fig. 9. In decode, KTransformers benefits from AVX-optimized kernels on Intel platforms, yet PDScope further improves throughput via systematic overhead reduction and adaptive load balancing, validating that phase-aware scheduling remains effective even when inter-request parallelism is available.

8.7 The Lookahead Depth of AdaptSched

A key question for AdaptSched's cross-layer scheduling is whether looking ahead only one layer (i.e., layers l and $l+1$) sufficiently captures the available optimization potential, or if a longer horizon (e.g., layers l, l+1, and l+2) is necessary. To investigate this, we compared a two-layer lookahead strategy against the default one-layer lookahead.

The two-layer strategy faces a practical limitation. If layers l and $l+1$ are both dominated by hot experts under cross-layer visibility, while layer $l+2$ is dominated by cold experts, the CPU has insufficient work to remain busy, and PCIe has little idle time to prefetch experts for layer $l+2$. In this case, the two-layer strategy must still degrade to one-layer lookahead to ensure optimal expert placement for layers l and $l+1$, rather than leaving the CPU idle across both layers. In practice, the two-layer strategy yields tangible benefits only when layers l and $l+1$ are both dominated by cold experts and layer $l+2$ by hot experts—a scenario where prefetching for layer $l+2$ can be effectively overlapped with CPU-bound computation of the preceding two layers. Furthermore, two-layer lookahead requires maintaining prefetch caches for experts across three layers simultaneously, imposing additional memory pressure.

Experiments on an A100 GPU with 4096-input and 64-output tokens show that the throughput difference between one- and two-layer lookahead is only 1.1%–2.3% for prefill and 0.8%–5.4% for decode across the four models. These results indicate that in long-context scenarios, one-layer lookahead effectively resolves the core scheduling questions—"whether to prefetch" and "how many to prefetch"—and delivers near-optimal performance.